Program Execution Basics

What is a computer program?

>> A computer program is a collection of instructions that performs a specific task when executed by a computer. A computer requires programs to function. A computer program is usually written by a computer programmer in a **programming language**. From the program in its human-readable form of **source code**, a compiler can derive machine code—a form consisting of instructions that the computer can directly execute. Alternatively, a computer program may be executed with the aid of an interpreter.

Compiler:

>> A compiler is a special program that processes statements written in a particular programming language and turns them into machine language or "code" that a computer's processor uses. Typically, a programmer writes language statements in a language such as Pascal or C one line at a time using an editor. The file that is created contains what are called the source statements or source code.

Interpreter:

>> An interpreter is a computer program that is used to directly execute program instructions written using one of the many high-level programming languages. The interpreter transforms the high-level program into an intermediate language that it then executes, or it could parse the high-level source code and then performs the commands directly, which is done line by line or statement by statement.

Assembler: Converts intermediate assembly code into a machine code.

The difference between an interpreter and a compiler is given below:

	Interpreter	**Compiler**
1	Translates program one statement at a time.	Scans the entire program and translates it as a whole into machine code.
2	It takes less amount of time to analyze the source code but the overall execution time is slower.	It takes large amount of time to analyze the source code but the overall execution time is comparatively faster.
3	No intermediate object code is generated, hence are memory efficient.	Generates intermediate object code which further requires linking, hence requires more memory.
4	Continues translating the program until the first error is met, in which case it stops. Hence debugging is easy.	It generates the error message only after scanning the whole program. Hence debugging is comparatively hard.
5	Programming language like **Python**, **Ruby** use interpreters.	Programming language like **C**, **C++** use compilers.



Figure: Compiler



Figure: Interpreter

The different kinds of files in C programming

Compiling C programs requires you to work with four kinds of files:

a) **Regular source code files:** These files contain function definitions, and have names which end in ".c" by convention.
b) **Header files:** These files contain function declarations (also known as function prototypes) and various preprocessor statements (see below). They are used to allow source code files to access externally-defined functions. Header files end in ".h" by convention.
c) **Object files:** These files are produced as the output of the compiler. They consist of function definitions in binary form, but they are not executable by themselves. Object files end in ".o" by convention, although on some operating systems (e.g. Windows, MS-DOS), they often end in ".obj".
d) **Binary executable:** These are produced as the output of a program called a "linker". The linker links together a number of object files to produce a binary file which can be directly executed. Binary executables have no special suffix on Unix operating systems, although they generally end in ".exe" on Windows.

The Four Stages of Compiling a C Program

Knowing how compilation works can be very helpful both when writing code and debugging. Compiling a C program is a multi-stage process. At an overview level, the process can be split into four separate stages:

a) Preprocessing

b) Compilation

c) Assembly

d) Linking

Traditional C compilers orchestrate this process by invoking (calling) other programs to handle each stage.

```c
/*
 * "Hello, World!": A classic.
 */

#include <stdio.h>

int
main(void)
{
        puts("Hello, World!");
        return 0;
}
```

A) Preprocessing

The first stage of compilation is called preprocessing. In this stage, lines starting with a "**#**" character are interpreted by the preprocessor as preprocessor commands. These commands form a simple macro language with its own syntax and semantics. This language is used to reduce repetition in source code by providing functionality to inline files, define macros and to conditionally omit code.

To print the result of the preprocessing stage, pass the -E option to cc:

```
cc -E hello_world.c
```

B) Compilation

The second stage of compilation is confusingly enough called compilation. In this stage, the preprocessed code is translated to assembly instructions specific to the target processor architecture. These form an intermediate human readable language.

The existence of this step allows for C code to contain inline assembly instructions and for different assemblers to be used.

Some compilers also supports the use of an integrated assembler, in which the compilation stage generates machine code directly, avoiding the overhead of generating the intermediate assembly instructions and invoking the assembler.

To save the result of the compilation stage, pass the -S option to cc:

```
cc -S hello_world.c
```

C) Assembly:

During the assembly stage, an assembler is used to translate the assembly instructions to machine code, or *object code*. The output consists of actual instructions to be run by the target processor.

To save the result of the assembly stage, pass the `-c` option to `cc`:

```
cc -c hello_world.c
```

Running the above command will create a file named `hello_world.o`, containing the object code of the program. The contents of this file is in a binary format and can be inspected using **hexdump** or **od** by running either one of the following commands:

```
hexdump hello_world.o

od -c hello_world.o
```

D) Linking

The object code generated in the assembly stage is composed of machine instructions that the processor understands but some pieces of the program are out of order or missing. To produce an executable program, the existing pieces have to be rearranged and the missing ones filled in. This process is called linking.

The linker will arrange the pieces of object code so that functions in some pieces can successfully call functions in other pieces. It will also add pieces containing the

instructions for library functions used by the program. In the case of the “Hello, World!” program, the linker will add the object code for the puts function.

The result of this stage is the final executable program. When run without options, cc will name this file a.out. To name the file something else, pass the -o option to cc:

```
cc -o hello_world hello_world.c
```